

Laborator 6 - Paternurile de proiectare comportamentale: Strategy, Observer, Command, Memento și Iterator

Condiție: În acest laborator, studenții vor explora și implementa cinci **paternuri comportamentale** esențiale: **Strategy, Observer, Command, Memento și Iterator**. Aceste paternuri sunt utilizate pentru a separa logica aplicației de comportamentele specifice, oferind mai multă flexibilitate și modularitate.

Pași de realizare:

1. Strategy:

- Se va implementa un mecanism pentru a permite alegerea și schimbarea dinamică a unui algoritm.
- Se va defini o interfață comună pentru toate strategiile, iar clasele concrete vor implementa diferite variante ale algoritmului.
- Se va demonstra cum utilizarea paternului Strategy permite modificarea comportamentului aplicației fără a afecta codul existent.

Exemplu: Un sistem de criptare care permite selectarea diferitelor metode de criptare (AES, RSA, SHA) fără a modifica logica existentă.

2. Observer:

- Se va implementa un sistem în care un obiect (subiect) notifică automat mai multe obiecte dependente (observatori) despre schimbările de stare.
- Se va crea o interfață pentru observatori și un mecanism de notificare atunci când starea subiectului se schimbă.
- Se va demonstra utilizarea paternului Observer în aplicații ce necesită notificări asincrone sau actualizări în timp real.

Exemplu: Un sistem meteo care trimite notificări pentru schimbările de vreme către aplicațiile asociate (telefon, panouri publice, alerte).

3. Command:

- Se va separa emiterea unei cereri de execuția efectivă a acesteia.
- Se vor implementa comenzi care pot fi stocate și executate ulterior, inclusiv pentru funcționalități Undo/Redo.
- Se va demonstra cum paternul Command permite gestionarea comenzilor într-un mod flexibil și extensibil.

Exemplu: Un sistem smart home care permite stocarea comenzilor de aprindere/stingere a luminilor și le execută ulterior, conform unui program.

4. **Memento:**

- Se va implementa un mecanism de salvare și restaurare a stării unui obiect fără a expune detaliile interne ale acestuia.
- Se vor crea obiecte Memento care vor salva starea unui alt obiect și o vor restaura la nevoie.
- Se va demonstra utilizarea paternului Memento pentru funcționalități Save/Load.

Exemplu: Un editor de text care permite salvarea versiunilor anterioare ale unui document și revenirea la o versiune mai veche.

5. **Iterator:**

- Se va implementa un mecanism pentru parcurgerea secvențială a unei colecții de obiecte fără a expune detaliile interne ale structurii acesteia.
- Se va crea un iterator care va oferi metode standardizate pentru accesarea elementelor colecției.
- Se va demonstra utilizarea paternului Iterator pentru a naviga prin colecții de date complexe fără a expune structura internă.

Exemplu: Un playlist muzical care permite navigarea prin melodii, fără a expune structura internă a listei de redare.

Testare și Refactorizare:

- Implementarea testelor unitare pentru fiecare patern utilizat.
 - Refactorizarea codului pentru îmbunătățirea clarității și modularității.
-

Prezentarea rezultatului:

- Explicarea fiecărui patern implementat, beneficiile acestuia și modul în care a fost aplicat în proiect.
-